

# HTTP :

## SOURCE CODE :

```
import requests
import logging

# Configure logging
logging.basicConfig(
    filename="http_errors.log",
    level=logging.ERROR,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

BASE_URL = "https://httpbin.org"

def get_request():
    print("\n===== HTTP GET REQUEST =====")

    try:
        response = requests.get(
            f'{BASE_URL}/get',
            timeout=10
        )

        print("Request Method : GET")
        print("Status Code  :", response.status_code)
```

```

print("\nResponse Headers:")

for key, value in response.headers.items():
    print(f"{key}: {value}")

print("\nResponse Body:")
print(response.text[:2000])

except requests.RequestException as error:
    print("GET request failed:", error)
    logging.error("GET request failed: %s", error)

def post_request():
    print("\n===== HTTP POST REQUEST =====")

    data = {
        "name": "Student",
        "course": "Computer Networks",
        "protocol": "HTTP"
    }

    try:
        response = requests.post(
            f"{BASE_URL}/post",
            json=data,
            timeout=10
        )

        print("Request Method : POST")

```

```

print("Status Code  :", response.status_code)

print("\nResponse Headers:")
for key, value in response.headers.items():
    print(f"{key}: {value}")

print("\nResponse Body:")
print(response.text[:2000])

except requests.RequestException as error:
    print("POST request failed:", error)
    logging.error("POST request failed: %s", error)

if __name__ == "__main__":
    print("APPLICATION LAYER PROTOCOL - HTTP")

    get_request()
    post_request()

    print("\nHTTP program completed.")

```

OUTPUT :

```
PS C:\Users\Yash\Downloads\assignment_2\assignment_@> python http_program.py
APPLICATION LAYER PROTOCOL - HTTP
```

```
===== HTTP GET REQUEST =====
```

```
Request Method : GET
Status Code    : 200
```

```
Response Headers:
```

```
Date: Wed, 26 Aug 2026 16:04:04 GMT
Content-Type: application/json
Content-Length: 305
Connection: keep-alive
Server: gunicorn/19.9.0
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

```
Response Body:
```

```
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.34.2",
    "X-Amzn-Trace-Id": "Root=1-6a8f0e74-57e3af395624d56113bb8ce9"
  },
  "origin": "139.5.9.103",
  "url": "https://httpbin.org/get"
}
```

```
===== HTTP POST REQUEST =====
```

```
Request Method : POST
Status Code    : 200
```

```
Response Headers:
```

```
Date: Wed, 26 Aug 2026 16:04:05 GMT
Content-Type: application/json
Content-Length: 608
Connection: keep-alive
Server: gunicorn/19.9.0
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

```
Response Body:
```

```
{
  "args": {},
  "data": "{\\\"name\\\": \\\"Student\\\", \\\"course\\\": \\\"Computer Networks\\\", \\\"protocol\\\": \\\"HTTP\\\"}",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Content-Length": "70",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.34.2",
    "X-Amzn-Trace-Id": "Root=1-6a8f0e75-20b32c6f2b3f4502581217ac"
  },
  "json": {
    "course": "Computer Networks",
    "name": "Student",
    "protocol": "HTTP"
  },
  "origin": "14.139.194.115",
  "url": "https://httpbin.org/post"
}
```

```
HTTP program completed.
```

# SMTP :

SOURCE CODE :

```
import smtplib

import logging

from email.message import EmailMessage

from getpass import getpass


logging.basicConfig(
    filename="smtp_log.txt",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)


def main():

    smtp_server = "smtp.gmail.com"

    smtp_port = 587


    sender = input("Enter sender Gmail address: ")
    password = getpass("Enter Gmail App Password: ")
    recipient = input("Enter recipient email address: ")


    message = EmailMessage()
    message["Subject"] = "Computer Networks Lab - SMTP Test"
    message["From"] = sender
    message["To"] = recipient
    message.set_content(
        "This is a test email sent using Python SMTP.\n"
```

"Computer Networks Lab Assignment 2."

)

try:

print("Connecting to SMTP server...")

logging.info("Connecting to %s:%s", smtp\_server, smtp\_port)

with smtplib.SMTP(smtp\_server, smtp\_port, *timeout*=20) as server:

server.ehlo()

logging.info("EHLO sent")

print("Starting TLS...")

server.starttls()

server.ehlo()

logging.info("TLS connection established")

print("Logging in...")

server.login(sender, password)

logging.info("SMTP login successful")

print("Sending email...")

server.send\_message(message)

logging.info("Email sent successfully")

print("\nEmail sent successfully.")

print("Communication log saved to smtp\_log.txt")

except Exception as e:

logging.exception("SMTP operation failed")

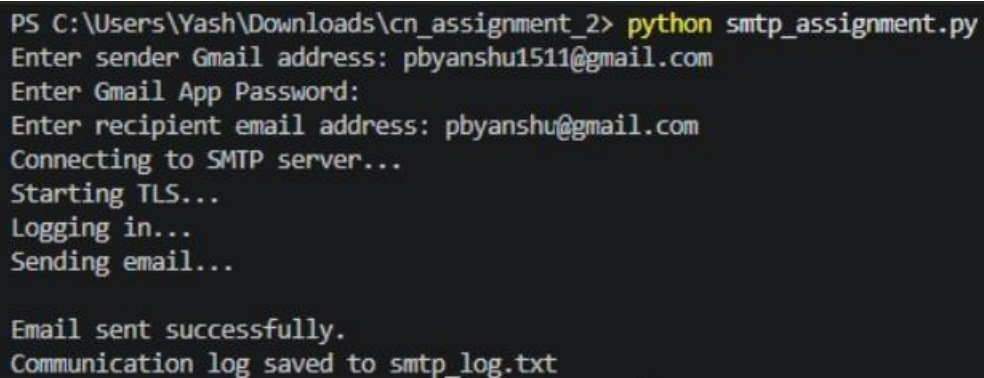
```
print("\nSMTP Error:", e)

print("Check your Gmail address and App Password.")

if __name__ == "__main__":

    main()
```

OUTPUT :



```
PS C:\Users\Yash\Downloads\cn_assignment_2> python smtp_assignment.py
Enter sender Gmail address: pbyanshu1511@gmail.com
Enter Gmail App Password:
Enter recipient email address: pbyanshu@gmail.com
Connecting to SMTP server...
Starting TLS...
Logging in...
Sending email...

Email sent successfully.
Communication log saved to smtp_log.txt
```

## FTP:

SOURCE CODE (Server):

```
from pyftplib.authorizers import DummyAuthorizer
from pyftplib.handlers import FTPHandler
from pyftplib.servers import FTPServer
from pathlib import Path
```

```
BASE_DIR = Path(__file__).parent / "ftp_root"
```

```
BASE_DIR.mkdir(exist_ok=True)
```

```
(BASE_DIR / "server_file.txt").write_text(
    "This file was created on the FTP server.\n"
```

```

"Computer Networks Lab Assignment 2.\n",
    encoding="utf-8"
)

authorizer = DummyAuthorizer()
authorizer.add_user("student", "12345", str(BASE_DIR), perm="elradfmwMT")

handler = FTPHandler
handler.authorizer = authorizer
server = FTPServer(("127.0.0.1", 2121), handler)
print("FTP server started.")
print("Host: 127.0.0.1")
print("Port: 2121")
print("Username: student")
print("Password: 12345")
print("Press Ctrl+C to stop the server.")
server.serve_forever()

```

SERVER RUNNING :

```

PS C:\Users\Yash\Downloads\cn_assignment_2> python ftp_server.py
FTP server started.
Host: 127.0.0.1
Port: 2121
Username: student
Password: 12345
Press Ctrl+C to stop the server.
[I 2026-08-26 20:09:17] concurrency model: async
[I 2026-08-26 20:09:17] masquerade (NAT) address: None
[I 2026-08-26 20:09:17] passive ports: None
[I 2026-08-26 20:09:17] >>> starting FTP server on 127.0.0.1:2121, pid=9940 <<<

```



SOURCE CODE (Client):

```
from ftplib import FTP

from pathlib import Path

HOST = "127.0.0.1"

PORT = 2121

USERNAME = "student"

PASSWORD = "12345"

def main():

    local_upload = Path("upload_test.txt")

    local_upload.write_text(

        "This file was uploaded using Python FTP.\n"

        "Computer Networks Lab Assignment 2.\n",

        encoding="utf-8"

    )

    downloaded = Path("downloaded_server_file.txt")

    print("==== FTP CLIENT =====")

    try:

        ftp = FTP()

        print("Connecting to FTP server...")

        ftp.connect(HOST, PORT, timeout=10)

        print("Connected.")

        ftp.login(USERNAME, PASSWORD)
```

```
print("Login successful.")
```

```
print("\n===== SERVER DIRECTORY =====")
```

```
ftp.retrlines("LIST")
```

```
print("\n===== UPLOAD =====")
```

```
with open(local_upload, "rb") as file:
```

```
    ftp.storbinary("STOR uploaded_test.txt", file)
```

```
print("Upload successful: uploaded_test.txt")
```

```
print("\n===== DIRECTORY AFTER UPLOAD =====")
```

```
ftp.retrlines("LIST")
```

```
print("\n===== DOWNLOAD =====")
```

```
with open(downloaded, "wb") as file:
```

```
    ftp.retrbinary("RETR server_file.txt", file.write)
```

```
print("Download successful: server_file.txt")
```

```
content = downloaded.read_text(encoding="utf-8")
```

```
print("\nDownloaded file content:")
```

```
print(content)
```

```
print("Content verification: SUCCESS")
```

```
ftp.quit()
```

```
print("\nFTP connection closed.")
```

```
except Exception as e:
```

```
    print("FTP Error:", e)
```

```
if __name__ == "__main__":  
    main()
```

OUTPUT CLIENT:

```
PS C:\Users\Yash\Downloads\cn_assignment_2> python ftp_client.py  
===== FTP CLIENT =====  
Connecting to FTP server...  
Connected.  
Login successful.  
  
===== SERVER DIRECTORY =====  
-rw-rw-rw-  1 owner  group          79 Aug 26 14:39 server_file.txt  
  
===== UPLOAD =====  
Upload successful: uploaded_test.txt  
  
===== DIRECTORY AFTER UPLOAD =====  
-rw-rw-rw-  1 owner  group          79 Aug 26 14:39 server_file.txt  
-rw-rw-rw-  1 owner  group          79 Aug 26 14:40 uploaded_test.txt  
  
===== DOWNLOAD =====  
Download successful: server_file.txt  
  
Downloaded file content:  
This file was created on the FTP server.  
Computer Networks Lab Assignment 2.  
  
Content verification: SUCCESS  
  
FTP connection closed.
```

## DNS :

SOURCE CODE :

```
import dns.resolver  
  
import logging
```

```
DOMAIN = "www.github.com"
```

```
OUTPUT_FILE = "dns_results.txt"
```

```
logging.basicConfig(
```

```
    filename="dns_queries.log",
```

```
    level=logging.INFO,
```

```
    format="%(asctime)s - %(levelname)s - %(message)s"
```

```
)
```

```
def query_record(domain, record_type):
```

```
    print(f"\n--- {record_type} RECORD ---")
```

```
    try:
```

```
        answers = dns.resolver.resolve(domain, record_type)
```

```
        for answer in answers:
```

```
            print(answer)
```

```
            logging.info(
```

```
                "%s %s -> %s",
```

```
                domain,
```

```
                record_type,
```

```
                answer
```

```
            )
```

```
        with open(OUTPUT_FILE, "a", encoding="utf-8") as file:
```

```
            file.write(
```

```
                f"{domain} {record_type} -> {answer}\n"
```

```
            )
```

```
except dns.resolver.NoAnswer:
```

```
    print(f"No {record_type} record found.")
```

```
    logging.warning(
```

```
        "No %s record found for %s",
```

```
        record_type,
```

```
        domain
```

```
    )
```

```
except dns.resolver.NXDOMAIN:
```

```
    print("Domain does not exist.")
```

```
    logging.error("NXDOMAIN: %s", domain)
```

```
except Exception as error:
```

```
    print(f"DNS query failed: {error}")
```

```
    logging.exception(
```

```
        "DNS query failed for %s %s",
```

```
        domain,
```

```
        record_type
```

```
    )
```

```
def main():
```

```
    print("===== DNS PROGRAM =====")
```

```
    print("Domain:", DOMAIN)
```

```
    # Clear previous result file
```

```
    with open(OUTPUT_FILE, "w", encoding="utf-8") as file:
```

```
        file.write(f"DNS Query Results for {DOMAIN}\n")
```

```

file.write("=" * 50 + "\n")

# Resolve IP address
print("\n===== IP ADDRESS =====")

try:
    ip_addresses = dns.resolver.resolve(DOMAIN, "A")

    for ip in ip_addresses:
        print("IP Address:", ip)

        with open(OUTPUT_FILE, "a", encoding="utf-8") as file:
            file.write(f"A -> {ip}\n")

        logging.info("A record for %s -> %s", DOMAIN, ip)

except Exception as error:
    print("Could not resolve IP address:", error)
    logging.exception("A record lookup failed.")

# Query different DNS records
query_record(DOMAIN, "A")
query_record(DOMAIN, "MX")
query_record(DOMAIN, "CNAME")

print("\nResults saved to:", OUTPUT_FILE)
print("DNS query log saved to: dns_queries.log")
print("\nDNS program completed.")

```

```
if __name__ == "__main__":  
    main()
```

OUTPUT:

```
PS C:\Users\Yash\Downloads\assignment_2\assignment_@> python dns_program.py  
===== DNS PROGRAM =====  
Domain: www.github.com  
  
===== IP ADDRESS =====  
IP Address: 20.207.73.82  
  
--- A RECORD ---  
20.207.73.82  
  
--- MX RECORD ---  
0 github-com.mail.protection.outlook.com.  
  
--- CNAME RECORD ---  
github.com.  
  
Results saved to: dns_results.txt  
DNS query log saved to: dns_queries.log  
  
DNS program completed.
```